

The Three Levels of LLM Evals: Unit Tests, Aligned Judges & A/B Testing

Build an evaluation-driven workflow — assertions on every change, human-aligned judges every week, and A/B tests on major releases.

For engineers building production LLM applications who want a systematic way to improve them instead of guessing · 27 July 2026

The one- or two-page version: what matters, the topics it covers, and every term defined. Read the full lesson for the explanations and worked examples.

[Watch the source video](#)[Read the full lesson](#)[Read the condensed version](#)

What matters

- ✓ Three core challenges define LLM development: not understanding your data at scale, the gap between what you want and what you can specify, and inconsistent behaviour on similar inputs.
- ✓ An eval is a systematic measurement of quality — a single metric for a specific aspect. One system has many evals.
- ✓ Evals are operationalised three ways: background monitoring, real-time guardrails, and improvement tooling for labelling and fine-tuning data.
- ✓ The analyze → measure → improve cycle is the working loop: collect and categorise failures, turn them into metrics, then change prompts, models, or architecture.
- ✓ Level 1 is unit tests — fast, cheap assertions run on every code and prompt change.
- ✓ Level 2 is human and model evaluation — systematic review run weekly or bi-weekly. Humans must come first; the LLM judge is aligned to them, never the reverse.
- ✓ Level 3 is A/B testing with real users, reserved for major releases because of its cost.
- ✓ The agreement score between human and model critique is the metric you optimise; meta-prompting can rewrite the judge prompt to raise it.
- ✓ Reference-based metrics compare to a known correct answer; reference-free metrics score qualities like tone or safety where many answers are valid.
- ✓ The four common mistakes are tool-first thinking, generic-metrics obsession, avoiding your data, and unaligned judges.

What it covers

- 01 **The questions evaluations exist to answer** 2:04
Every engineer building on LLMs eventually hits a wall of questions that logs alone cannot answer.
- 02 **The three core challenges of LLM development** 4:52
Understanding your data, closing the specification gap, and coping with inconsistent behaviour — these are what evaluation exists to address.
- 03 **What an eval is, and the analyze-measure-improve cycle** 9:01
An eval is a systematic measurement of one specific quality aspect; the working loop is collect failures, turn them into metrics, then improve.
- 04 **Level 1: unit tests as evals** 12:26

Fast, cheap Python assertions over structured output, run on every code and prompt change.

05 **Level 2: humans first, then an aligned model judge** 17:55

The common misconception is that you can hand evaluation straight to an LLM. You cannot instruct a model to review what you have not learned to review yourself.

06 **The alignment process and the agreement score** 22:47

Run model critique and human critique in parallel over the same rows, score the match, and iterate the judge prompt until agreement climbs.

07 **Level 3: A/B testing with real users** 31:46

The trickiest and most expensive level — compare prompts, models, or whole workflows by their effect on real business outcomes.

08 **Reference-based versus reference-free metrics** 34:29

When you know the correct answer, testing is easy; when many answers are valid, you score qualities instead.

09 **Four mistakes, and the principles that avoid them** 37:12

Tool-first thinking, generic-metrics obsession, avoiding your data, and unaligned judges — with looking at data as the highest-value habit.

Glossary

Eval — A systematic measurement of quality in an AI system; a single eval measures one specific aspect of performance.

Analyze–measure–improve — The evaluation lifecycle: collect and categorise failures, turn insights into quantitative metrics, then refine prompts, models, or architecture.

Assertion — A Python statement that passes silently when its condition is true and raises an `AssertionError` when false, used as the basis of level 1 evals.

Meta-prompting — Asking a strong model to rewrite your judge prompt by showing it the human-model disagreements and the current prompt, so it encodes what the human cares about.

Reference-free metric — A measurement of a quality where many outputs are valid — tone, length, hallucination, format compliance, safety.

Raw event — The exact input payload your workflow received, captured and stored so it can later be replayed as a test fixture.

Evaluation-driven workflow — A development process where improvement is systematic rather than accidental, because every change is measured against a fixed set of evals.

Specification gap — The distance between what you want a system to do and what you can actually express in prompts and code.

Agreement score — The fraction of examples where a human's critique outcome matches the LLM judge's outcome; the metric you iterate the judge prompt to improve.

Reference-based metric — A measurement made against a known correct answer — exact match, semantic similarity, code execution, or schema validation.

Guardrail — An eval operationalised in real time to block a bad output before it reaches the user.

Drift — The gradual change over time in what 'good' means for your system, caused by prompt changes, data changes, new features, or different users.

Explanations are AI-generated. Check anything you intend to rely on.